

Ecole Supérieure Polytech Diamniadio - UAM

Ingénierie des Systèmes d'Information et Données



Rapport de Projet - Mise en Place d'un Pipeline CI/CD

GitHub · Jenkins · Docker · Kubernetes · Prometheus · Grafana

Présenté par : **Mr. Maguette DIOP**

Superviseur : **Dr Massamba LO**

1. Introduction

Ce rapport décrit la conception et la mise en œuvre d'un pipeline complet d'Intégration Continue et de Déploiement Continu (CI/CD) pour l'application **FleetCom**, déployée sur une infrastructure cloud basée sur AWS.

L'objectif du projet est d'automatiser l'ensemble du cycle de vie de l'application, depuis l'intégration du code source jusqu'à son déploiement en production. L'automatisation permet de réduire les interventions manuelles, d'améliorer la fiabilité des déploiements et d'accélérer la livraison de nouvelles versions.

Le pipeline implémenté couvre plusieurs étapes essentielles du processus DevOps :

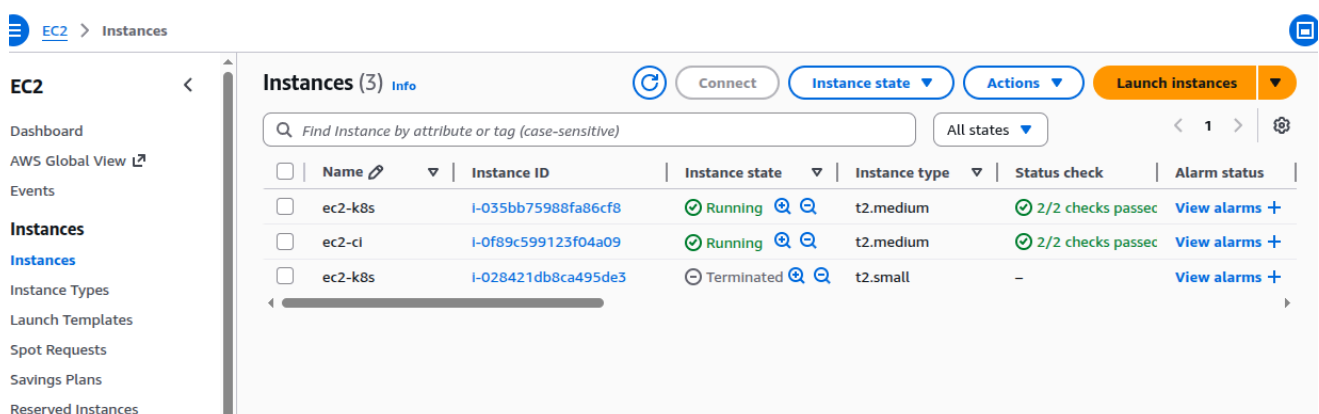
- développement du code et versionnement dans un dépôt GitHub
- déclenchement automatique du pipeline Jenkins via un webhook
- construction de l'image Docker de l'application
- publication de l'image dans le registre Docker Hub
- déploiement automatique sur un cluster Kubernetes
- collecte et visualisation des métriques avec Prometheus et Grafana

Ainsi, chaque modification du code source peut être automatiquement construite, déployée et supervisée dans un environnement cloud.

2. Architecture du Projet

2.1 Vue d'ensemble

L'architecture repose sur deux instances AWS EC2 ayant des rôles distincts mais complémentaires.



Aws-Ec2-Instances-List-Page-Showing-Ec2-Ci-And-Ec2-K8s-Instances-Up.Png

Figure 1 — Les deux instances EC2 (ec2-ci et ec2-k8s) actives sur AWS

Instance	Rôle et contenu
ec2-ci	Serveur d'intégration continue. Il héberge Jenkins (installé nativement), Docker pour la construction des images, et kubectl configuré pour piloter le cluster Kubernetes distant.
ec2-k8s	Serveur de déploiement. Il héberge Minikube (driver Docker), les pods de l'application FleetCom, ainsi que la stack de monitoring Prometheus et Grafana.

Cette séparation permet d'isoler les tâches d'intégration continue de l'environnement de production.

2.2 Schéma de la pipeline

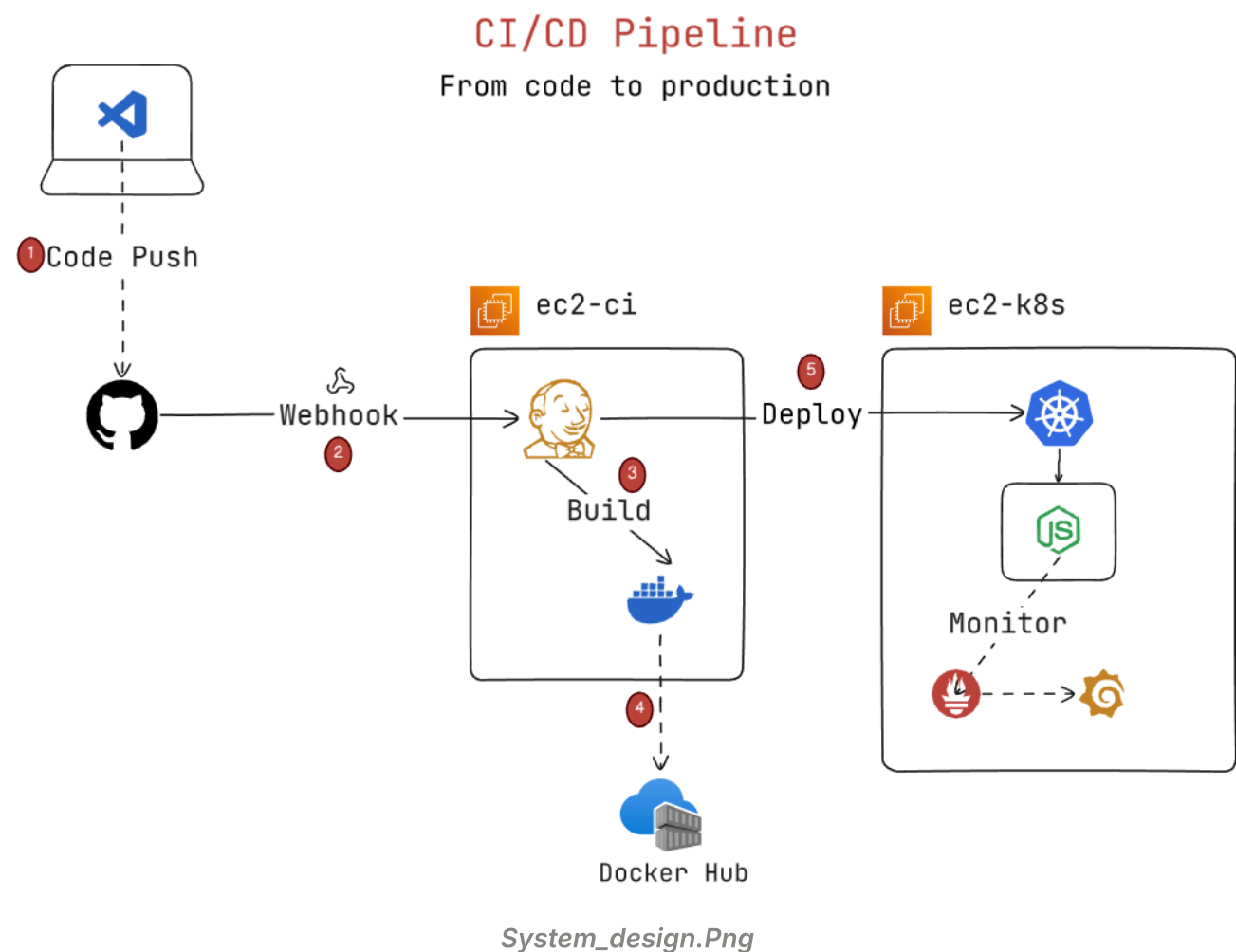


Figure 2 — Schéma CI/CD : Code → GitHub → Jenkins → Docker Hub → Kubernetes → Monitor

Le flux de fonctionnement du pipeline est le suivant :

- ① Le développeur effectue un git push vers le dépôt GitHub
- ② GitHub déclenche un **webhook HTTP** vers Jenkins sur l'instance ec2-ci
- ③ Jenkins clone le dépôt, construit l'image Docker et la publie sur Docker Hub
- ④ Jenkins met à jour le déploiement Kubernetes sur ec2-k8s via **kubectl set image**
- ⑤ Prometheus collecte les métriques des pods et Grafana les visualise

2.3 Technologies utilisées

Technologie	Usage dans le projet
GitHub	Hébergement du code source et déclencheur webhook
Jenkins	Serveur CI/CD installé sur ec2-ci (port 8080)
Docker	Containerisation de l'application FleetCom
Docker Hub	Registre des images Docker (jiem117/fleetcom)
Minikube	Cluster Kubernetes local sur ec2-k8s (driver Docker)
kubectl	Client CLI Kubernetes configuré sur ec2-ci
Helm	Gestionnaire de packages Kubernetes
Prometheus	Collecte des métriques du cluster
Grafana	Visualisation des métriques

3. Configuration de l'Infrastructure

3.1 Instance ec2-ci — Serveur Jenkins

L'instance **ec2-ci** héberge Jenkins installé directement sur le système d'exploitation, ce qui évite les problèmes d'accès au socket Docker lorsque Jenkins est exécuté dans un conteneur.

Les composants suivants ont été installés :

- OpenJDK 21 (pré-requis Jenkins)
- Jenkins LTS depuis le dépôt officiel
- Docker CE pour la construction des images
- kubectl pour l'administration du cluster Kubernetes

Commandes exécutées sur **ec2-ci** :

```
# Installation de Jenkins
```

```
sudo apt install fontconfig openjdk-21-jre
```

```
sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc  
https://pkg.jenkins.io/debian-stable/jenkins.io-2026.key
```

```
sudo apt install jenkins
```

```
# Installation de Docker
```

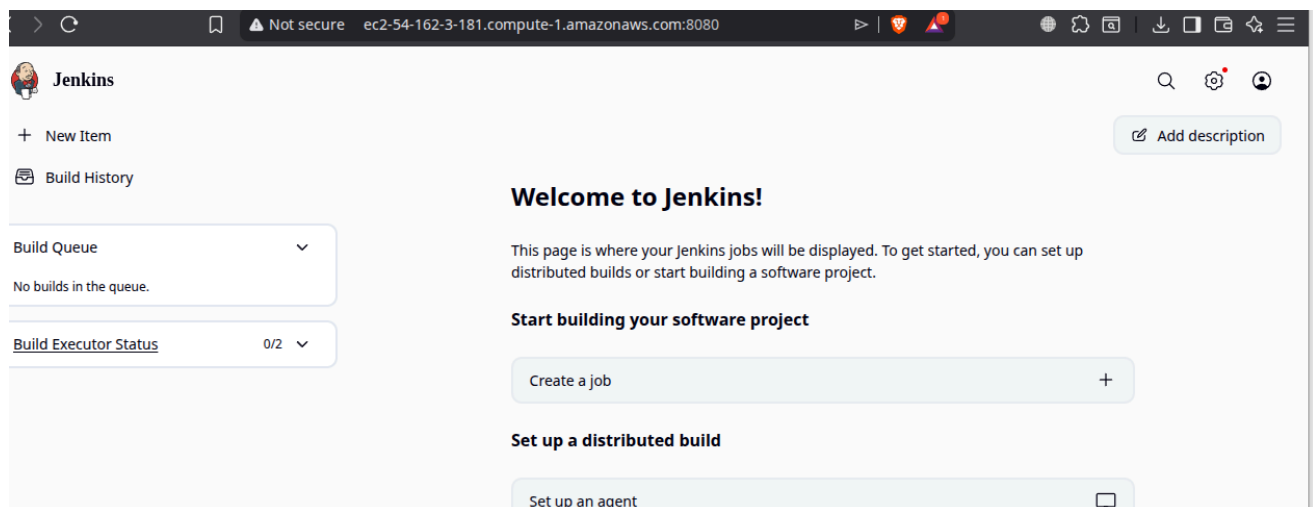
```
sudo apt install docker-ce docker-ce-cli containerd.io
```

```
sudo usermod -aG docker jenkins
```

```
# Installation de kubectl
```

```
sudo curl -LO "https://dl.k8s.io/release/.../kubectl"
```

```
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
```



Jenkins-Welcome-Page.Png

Figure 3 — Interface Jenkins opérationnelle sur ec2-ci (port 8080)

3.2 Instance ec2-k8s — Serveur Kubernetes

L'instance **ec2-k8s** exécute le cluster Kubernetes basé sur **Minikube** avec le driver Docker. Minikube crée un cluster Kubernetes à nœud unique fonctionnant dans un conteneur Docker.

Commandes exécutées sur **ec2-k8s** :

```
# Installation de Minikube

curl -LO
https://storage.googleapis.com/minikube/releases/latest/minikube_latest_amd64.deb

sudo dpkg -i minikube_latest_amd64.deb
```

```
# Démarrage avec support IP publique

minikube start --apiserver-ips=54.165.90.222
```

```
# Vérification du cluster

kubectl get nodes
```

Figure 4 — kubectl get nodes -o wide depuis ec2-ci et ec2-k8s

3.3 Connexion ec2-ci → ec2-k8s (kubeconfig)

Pour permettre à Jenkins d'utiliser **kubectl** afin de piloter le cluster distant, le fichier **kubeconfig** est transféré depuis l'instance ec2-k8s vers ec2-ci.

```
# Sur ec2-k8s : générer un kubeconfig avec les certificats

kubectl config view --flatten --minify > ~/.kube/config-flat
```

```
# Sur ec2-ci : ajouter le kubeconfig pour l'utilisateur jenkins

sudo cp ~/.kube/config-flat /var/lib/jenkins/.kube/config

sudo chown -R jenkins:jenkins /var/lib/jenkins/.kube
```

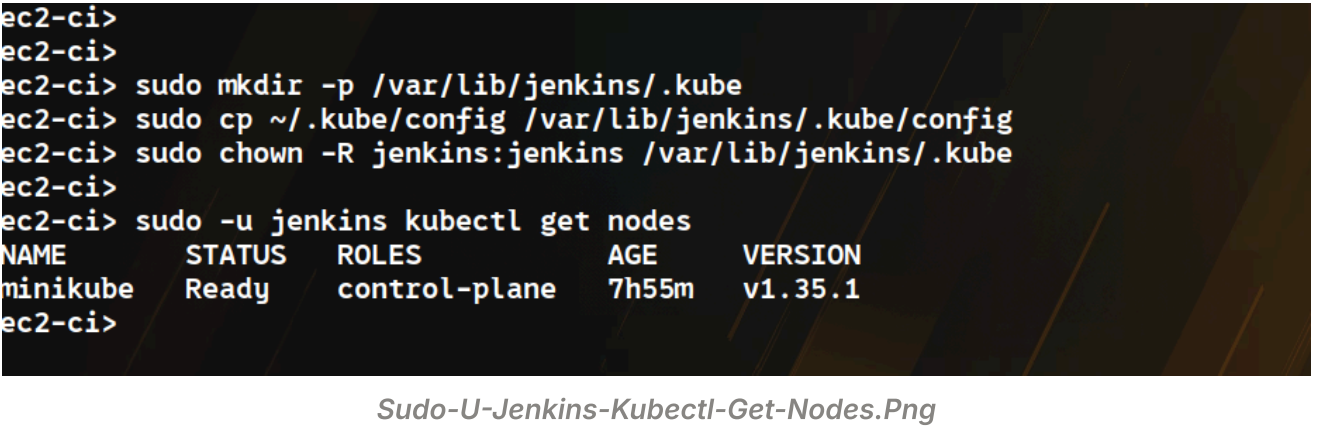


Figure 5 — kubectl get nodes fonctionnel avec l'utilisateur jenkins

3.4 Security Groups AWS

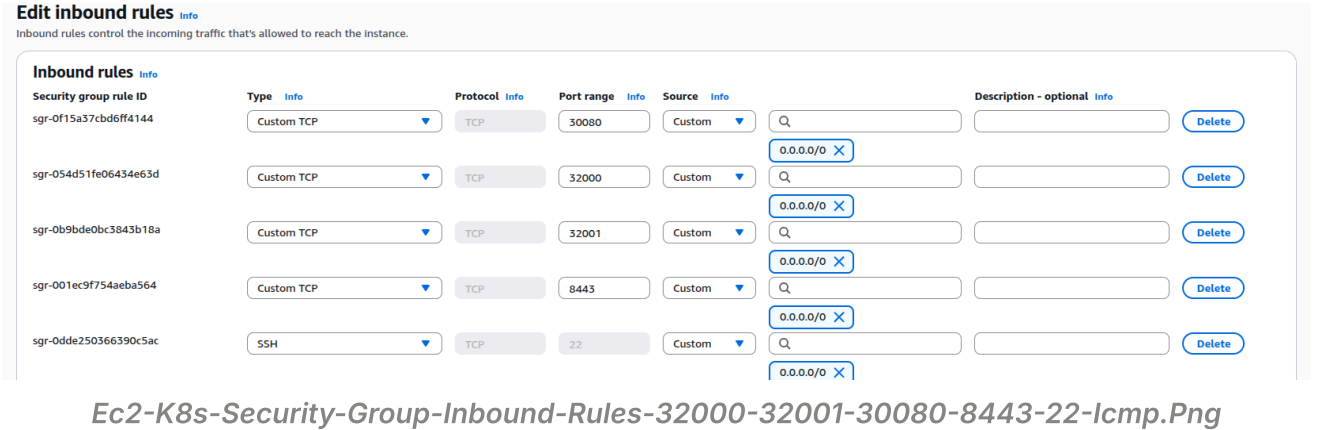
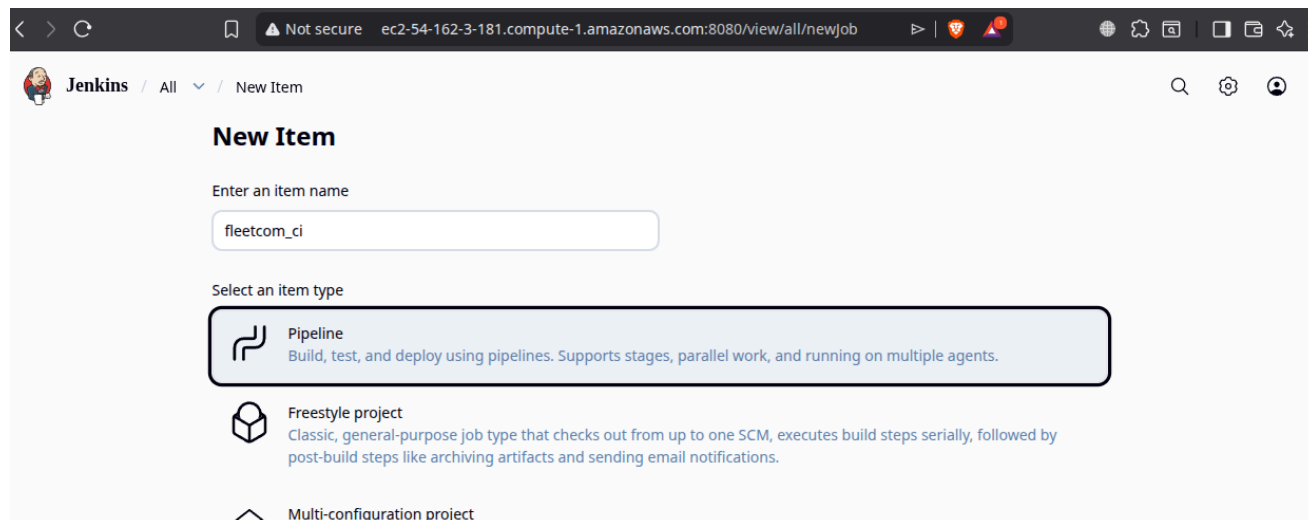


Figure 6 — Règles inbound du Security Group ec2-k8s

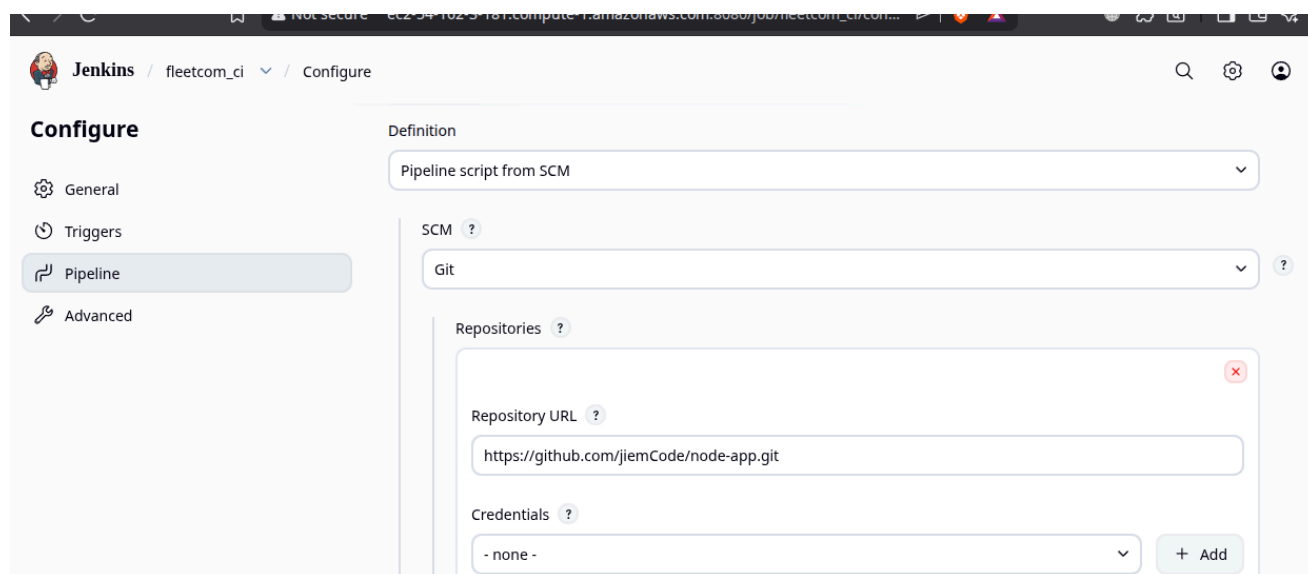
Port	Usage
22	Accès SSH
8443	API Kubernetes
30080	NodePort FleetCom
32000	Grafana
32001	Prometheus

Le job Jenkins est configuré en **Pipeline** et récupère directement le **Jenkinsfile** depuis le dépôt GitHub.



Fleetcom-Ci-Jenkins-Pipeline-Creation.Png

Figure 7 — Création du job Pipeline FleetCom dans Jenkins

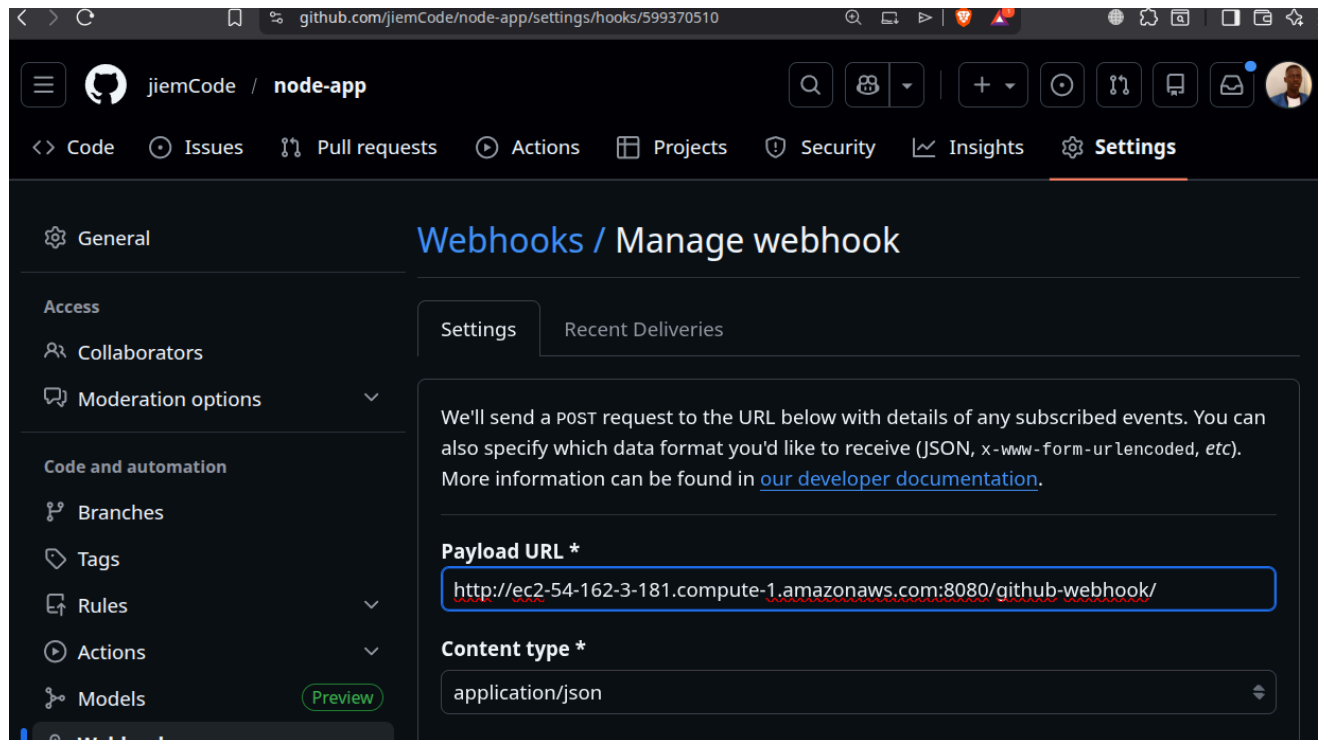


Fleetcom-Ci-Pipeline-Setup-On-Jenkins-Scm.Png.Png

Figure 8 — Configuration SCM : récupération du Jenkinsfile depuis GitHub

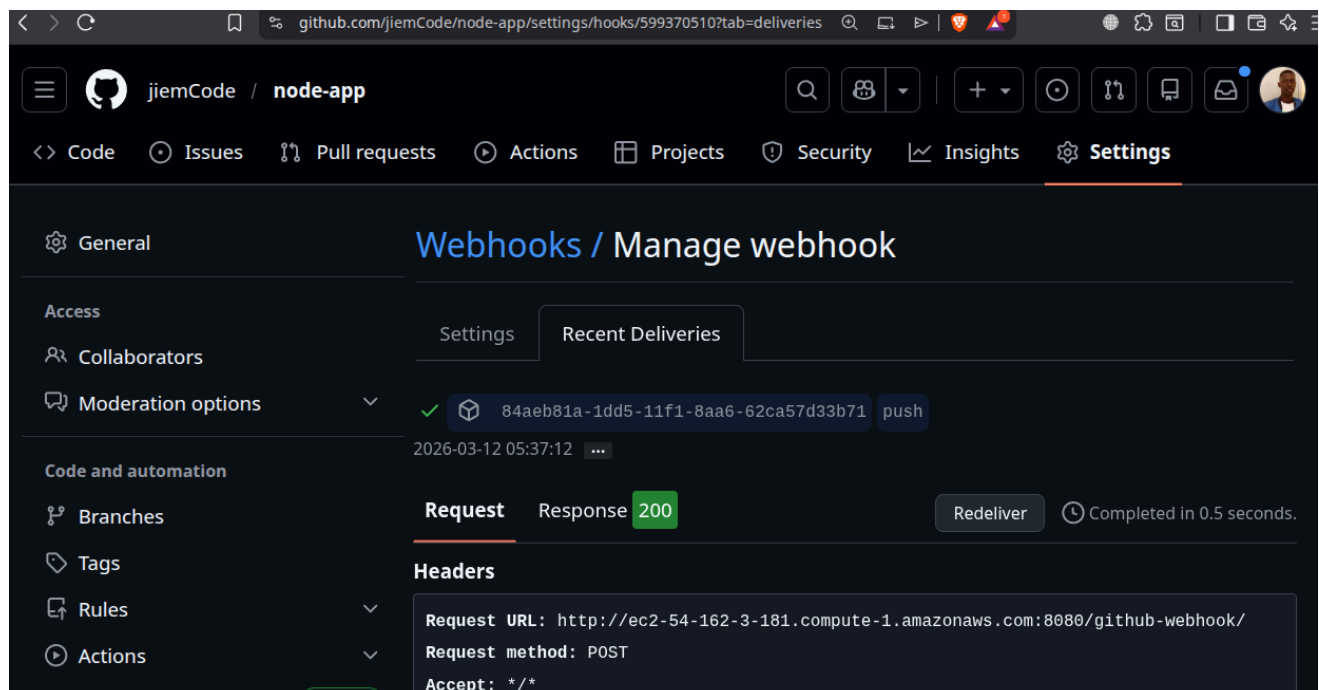
4.2 Déclenchement automatique via Webhook

Le pipeline est déclenché automatiquement après chaque **push** sur GitHub grâce à un webhook.



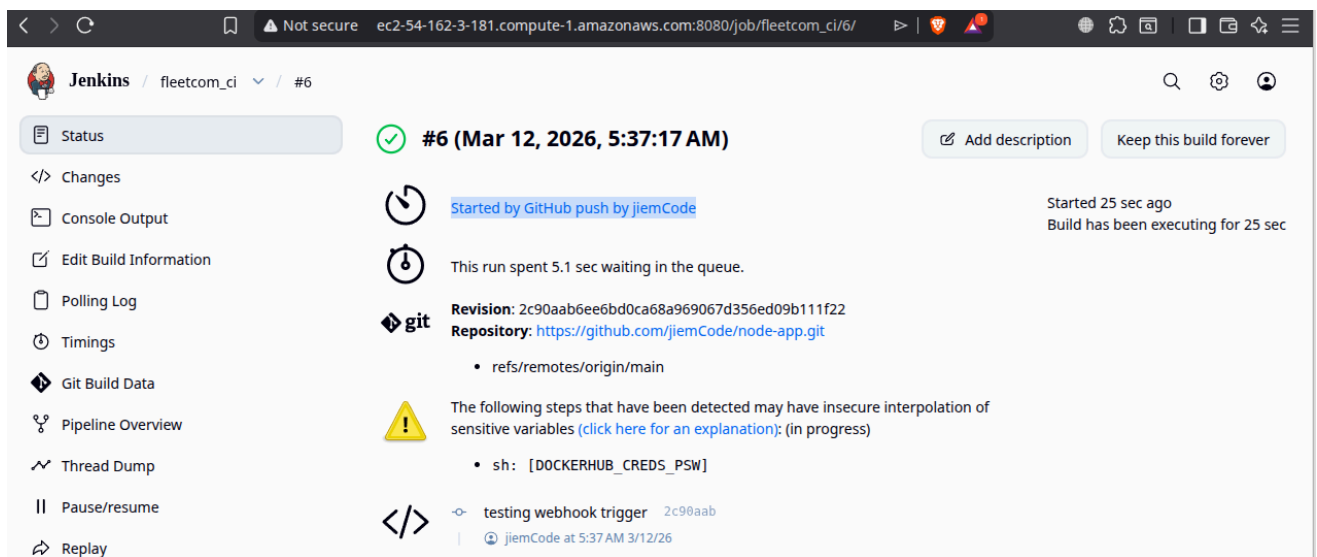
Github-Repo-Webhook-Configuration.Png

Figure 9 — Configuration du webhook GitHub vers Jenkins



Github-Webhook-Delivers-List.Png

Figure 10 — Historique des livraisons webhook (statut 200)



Jenkins-Job-Trigger-By-Webhook.Png

Figure 11 — Activation du trigger Jenkins

4.3 Jenkinsfile — Définition du Pipeline

Le pipeline est défini dans le fichier **Jenkinsfile** suivant :

```
pipeline {

    agent any

    environment {

        DOCKERHUB_USER = "jiem117"

        APP_NAME = "fleetcom"

        K8S_NAMESPACE = "default"

        DOCKER_IMAGE = "${DOCKERHUB_USER}/${APP_NAME}"

        IMAGE_TAG = "${BUILD_NUMBER}"

        DOCKERHUB_CREDS = credentials('dockerhub-credentials')

    }

    stages {

        stage('Checkout') {

            steps { checkout scm }

        }

        stage('Build Docker Image') {

            steps {

                sh "docker build -t ${DOCKER_IMAGE}:${IMAGE_TAG} ."

                sh "docker tag ${DOCKER_IMAGE}:${IMAGE_TAG} ${DOCKER_IMAGE}:latest"

            }

        }

        stage('Push to Docker Hub') {

            steps {

                sh "echo ${DOCKERHUB_CREDS_PSW} | docker login -u ${DOCKERHUB_CREDS_USR} --password-stdin"

                sh "docker push ${DOCKER_IMAGE}:${IMAGE_TAG}"

                sh "docker push ${DOCKER_IMAGE}:latest"

            }

        }

    }

}
```

```

}

stage('Deploy to Kubernetes') {

  steps {

    sh """kubectl set image deployment/${APP_NAME} \

    ${APP_NAME}=${DOCKER_IMAGE}:${IMAGE_TAG} -n ${K8S_NAMESPACE} \

    || kubectl apply -f k8s/ -n ${K8S_NAMESPACE}"""

  }

}

stage('Verify Rollout') {

  steps {

    sh "kubectl rollout status deployment/${APP_NAME} --timeout=120s"

  }

}

}

post {

  always { sh 'docker logout || true' }

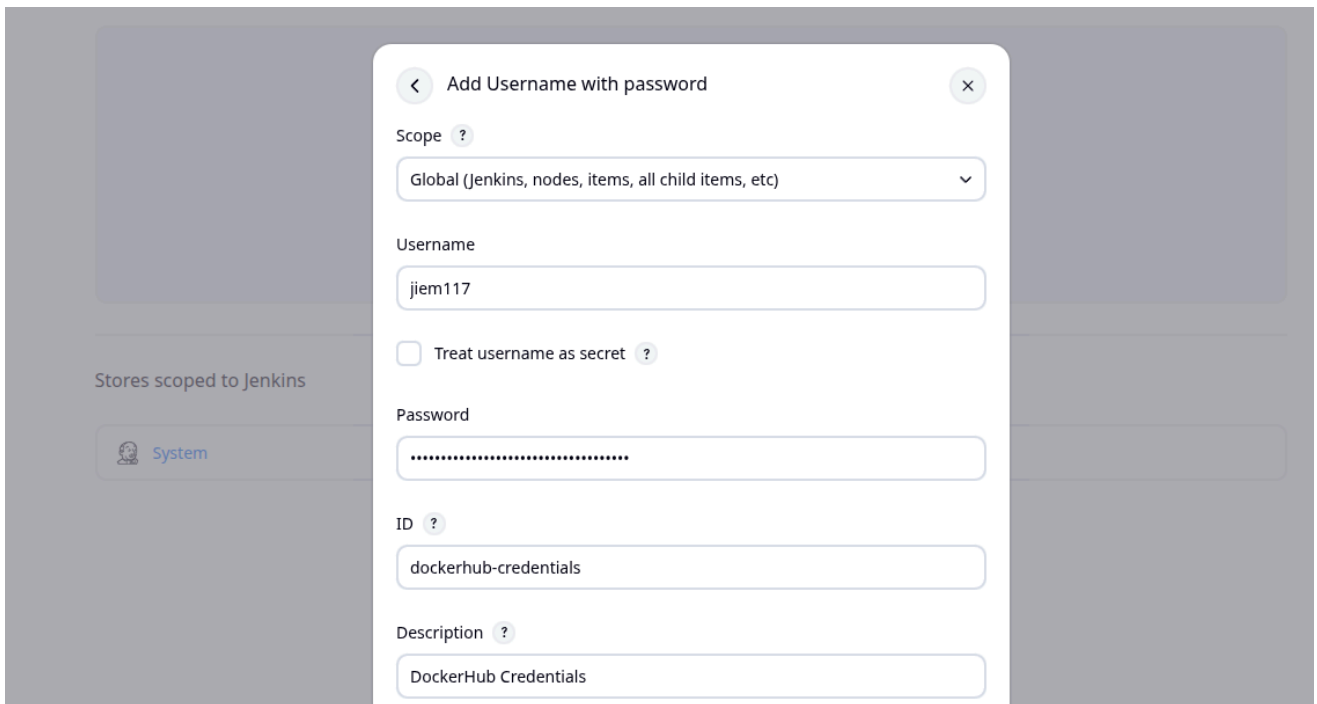
}

}

```

4.4 Gestion des credentials

Les identifiants Docker Hub sont stockés dans Jenkins de manière sécurisée et injectés dans le pipeline au moment de l'exécution.

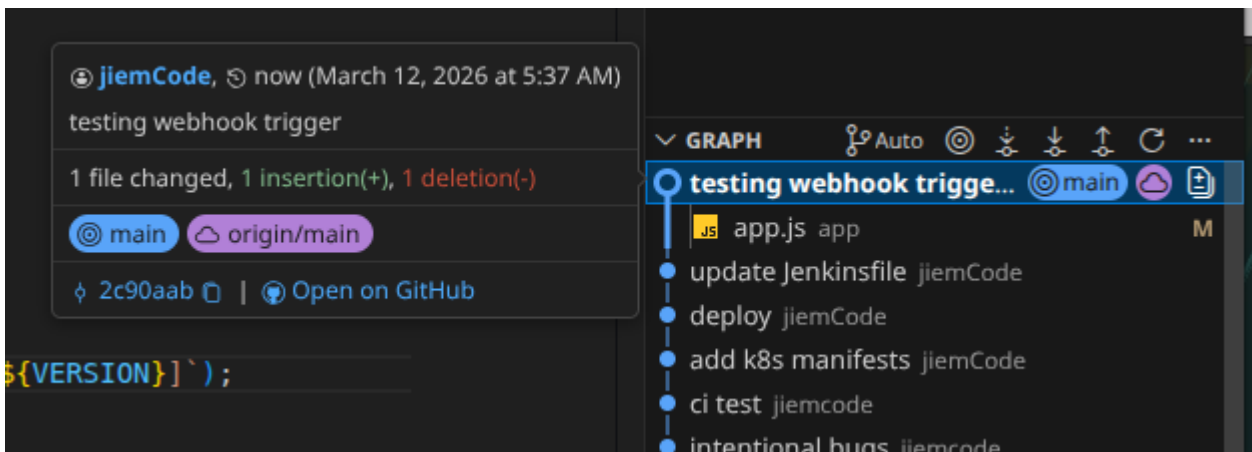


Dockerhub-Credential-Modal-On-Jenkins-Secret.Png

Figure 12 — Credentials Docker Hub configurés dans Jenkins

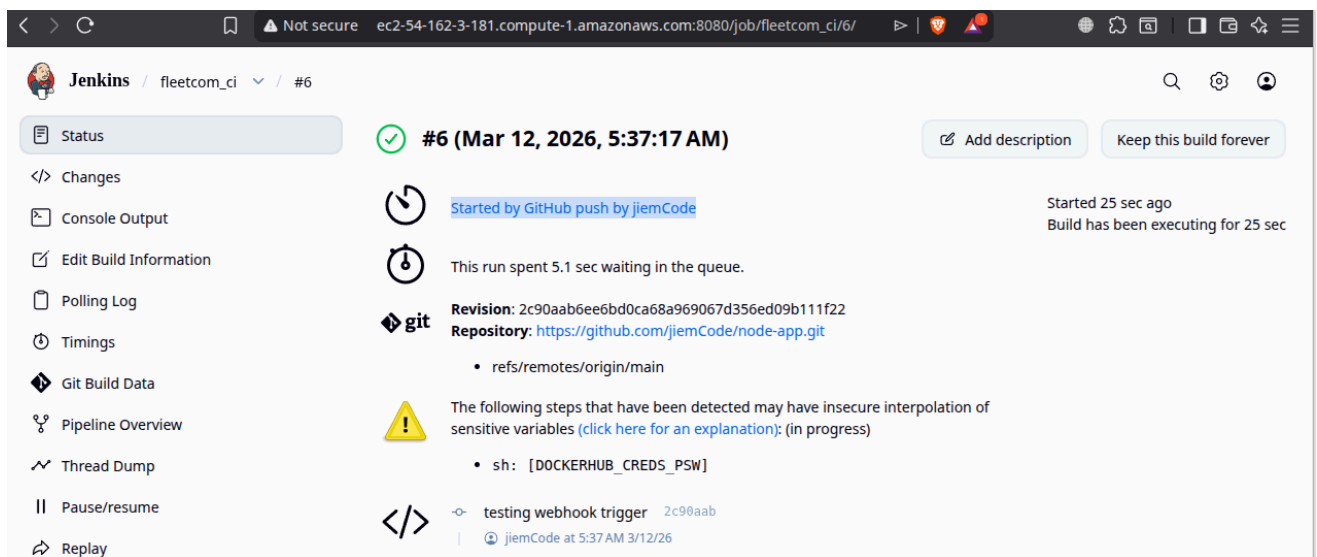
4.5 Exécution du pipeline

Un commit sur GitHub déclenche automatiquement le pipeline.



Test-Commit-To-Trigger-Webhook.Png

Figure 13 — Commit de test



Jenkins-Job-Trigger-By-Webhook.Png

Figure 14 — Pipeline déclenché automatiquement

5. Déploiement Kubernetes

5.1 Manifestes Kubernetes

Deployment :

```
apiVersion: apps/v1

kind: Deployment

metadata:

  name: fleetcom

  namespace: default

spec:

  replicas: 2

  selector:

    matchLabels:

      app: fleetcom

  strategy:

    type: RollingUpdate

  rollingUpdate:

    maxSurge: 1

    maxUnavailable: 0

  template:

    spec:

      containers:

        - name: fleetcom

          image: jiem117/fleetcom:latest

          imagePullPolicy: Always

      ports:

        - containerPort: 3000
```

Service :

```
apiVersion: v1

kind: Service

metadata:

name: fleetcom-service

spec:

selector:

app: fleetcom

type: NodePort

ports:

- port: 80

targetPort: 3000

nodePort: 30080
```

5.2 Premier déploiement manuel

```
kubectl apply -f k8s/
```

```
ec2-ci>
ec2-ci>
ec2-ci> mkdir k8s
ec2-ci> nano k8s/deployment.yaml
ec2-ci> nano k8s/service.yaml
ec2-ci>
ec2-ci>
ec2-ci> kubectl apply -f k8s/
deployment.apps/fleetcom created
service/fleetcom-service created
ec2-ci>
ec2-ci>
ec2-ci> k get deployments
NAME          READY    UP-TO-DATE    AVAILABLE    AGE
fleetcom      2/2      2             2            20s
ec2-ci> k get services
NAME          TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
fleetcom-service  NodePort    10.108.6.243  <none>         80:30080/TCP     25s
kubernetes     ClusterIP   10.96.0.1     <none>         443/TCP          8h
ec2-ci> █
```

First-Manual-Deploy-Kubectl-Apply-F-K8s.Png

Figure 15 — Premier déploiement Kubernetes

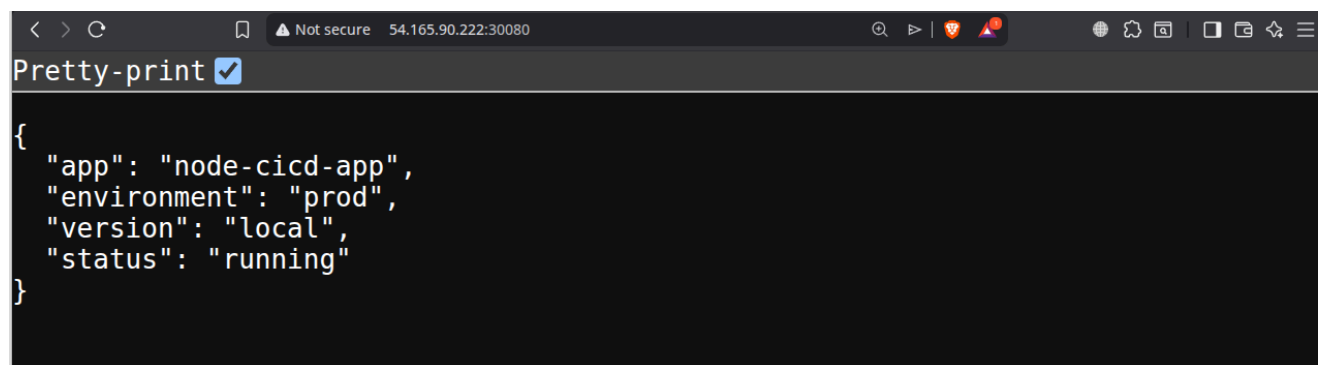
5.3 Pods en exécution

```
ec2-k8s> k get pods
NAME                                READY   STATUS    RESTARTS   AGE
fleetcom-54b4bf96b8-2hhfv          1/1     Running   0           2m34s
fleetcom-54b4bf96b8-lp8gn          1/1     Running   0           2m16s
ec2-k8s> k get pods -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP            NODE
fleetcom-54b4bf96b8-2hhfv          1/1     Running   0           2m50s  10.244.0.10   minikube
fleetcom-54b4bf96b8-lp8gn          1/1     Running   0           2m32s  10.244.0.11   minikube
ec2-k8s>
```

Kubectl-Get-Pods.Png

Figure 16 — Pods FleetCom en état Running

5.4 Exposition publique



A screenshot of a web browser window. The address bar shows the URL `54.165.90.222:30080`. The page content displays a JSON object: `{ "app": "node-cicd-app", "environment": "prod", "version": "local", "status": "running" }`. The browser's developer tools are open, showing the JSON response.

Demo-App-Exposed-Through-K8s-On-Port-30080.Png

Figure 17 — Application accessible sur

6. Monitoring — Prometheus et Grafana

6.1 Installation via Helm

```
ec2-k8s>
ec2-k8s> helm repo add prometheus-community https://prometheus-community.github.io/helm-c
"prometheus-community" has been added to your repositories
ec2-k8s>
ec2-k8s> helm repo update
Hang tight while we grab the latest from your chart repositories...
...Successfully got an update from the "prometheus-community" chart repository
Update Complete. *Happy Helming!*
ec2-k8s>
```

Helm-Repo-Add-Prometheus.Png

Figure 18 — Ajout du dépôt Helm

```

ec2-k8s>
ec2-k8s>
ec2-k8s> helm install monitoring prometheus-community/kube-prometheus-stack \
--namespace monitoring \
--create-namespace \
--set grafana.service.type=NodePort \
--set grafana.service.nodePort=32000 \
--set prometheus.service.type=NodePort \
--set prometheus.service.nodePort=32001
NAME: monitoring
LAST DEPLOYED: Thu Mar 12 05:43:06 2026
NAMESPACE: monitoring
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
kube-prometheus-stack has been installed. Check its status by running:
  kubectl --namespace monitoring get pods -l "release=monitoring"

Get Grafana 'admin' user password by running:

  kubectl --namespace monitoring get secrets monitoring-grafana -o jsonpath="{.data.admin-password}" | base64
Access Grafana local instance:

```

Helm-Install-Monitoring-Prometheus.Png

Figure 19 — Installation kube-prometheus-stack

```

helm repo add prometheus-community https://prometheus-
community.github.io/helm-charts

```

```

helm repo update

```

```

helm install monitoring prometheus-community/kube-prometheus-stack \

--namespace monitoring --create-namespace \

--set grafana.service.type=NodePort \

--set grafana.service.nodePort=32000 \

--set prometheus.service.type=NodePort \

--set prometheus.service.nodePort=32001

```

6.2 Pods de monitoring

```

ec2-k8s> kubectl get pods -n monitoring

```

NAME	READY	STATUS	RESTARTS	AGE
alertmanager-monitoring-kube-prometheus-alertmanager-0	2/2	Running	0	45s
monitoring-grafana-5fdb9874c9-7zx6q	3/3	Running	0	59s
monitoring-kube-prometheus-operator-69886c5966-bjzcg	1/1	Running	0	60s
monitoring-kube-state-metrics-f56d7b58b-6f795	1/1	Running	0	59s
monitoring-prometheus-node-exporter-khpfs	1/1	Running	0	59s
prometheus-monitoring-kube-prometheus-prometheus-0	1/2	Running	0	42s

Kubectl-Get-Pods--N-Monitoring.Png

Figure 20 — Pods du namespace monitoring

6.3 Récupération des credentials Grafana

```
ec2-k8s> kubectl get secret -n monitoring monitoring-grafana \
-o jsonpath="{.data.admin-password}" | base64 --decode
mrfe6FSt4iA2qeAapjzu3KLxYGlV3zLbuw40jiu7ec2-k8s>
```

Kubectl-Get-Secret-For-Grafana-Dashboard.Png

Figure 21 — Récupération du mot de passe Grafana

```
kubectl get secret -n monitoring monitoring-grafana -o jsonpath="
{.data.admin-password}" | base64 --decode
```

6.4 Tableaux de bord Grafana

Figure 23 — Dashboard Kubernetes

Figure 24 — Métriques CPU, mémoire et réseau

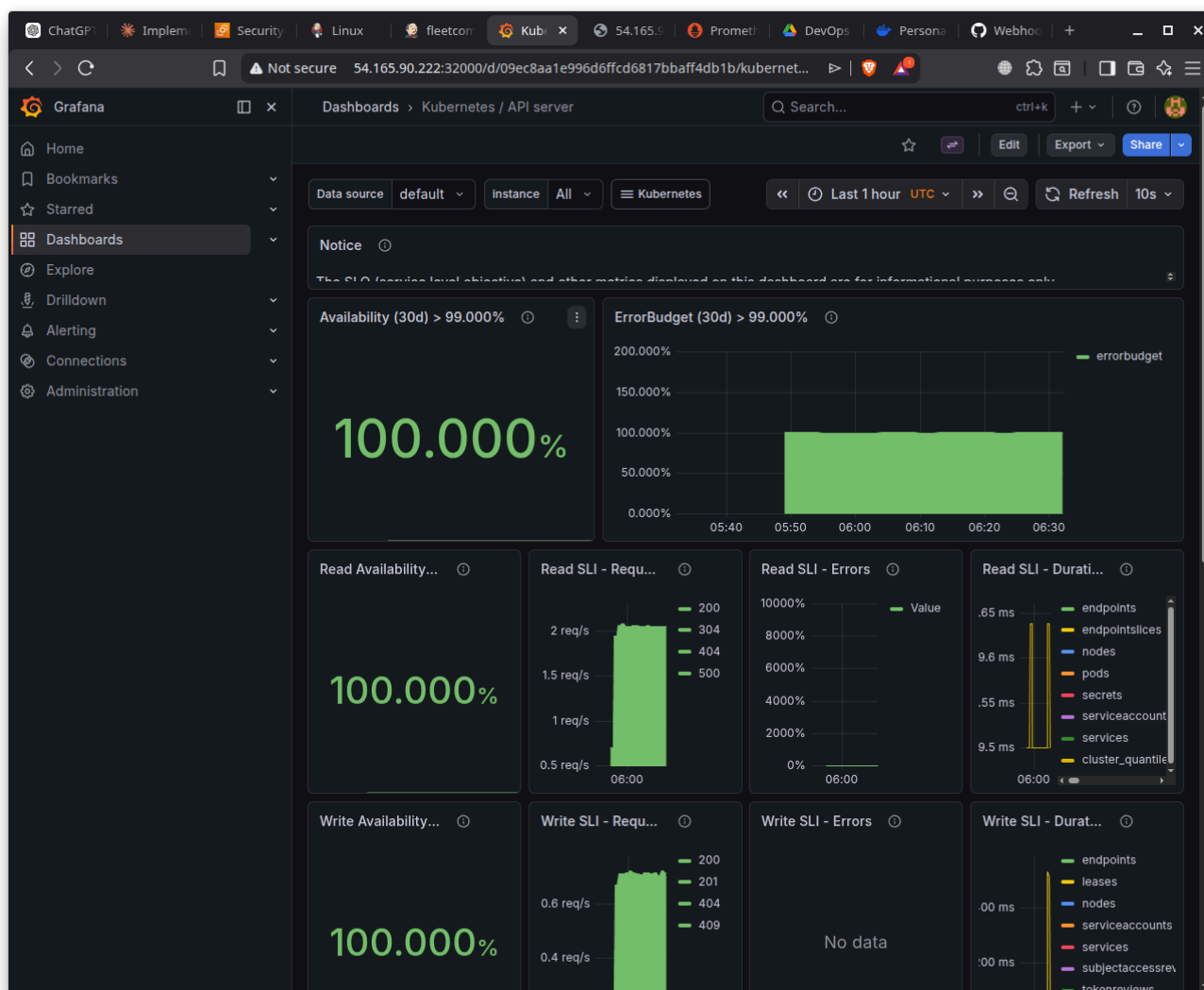
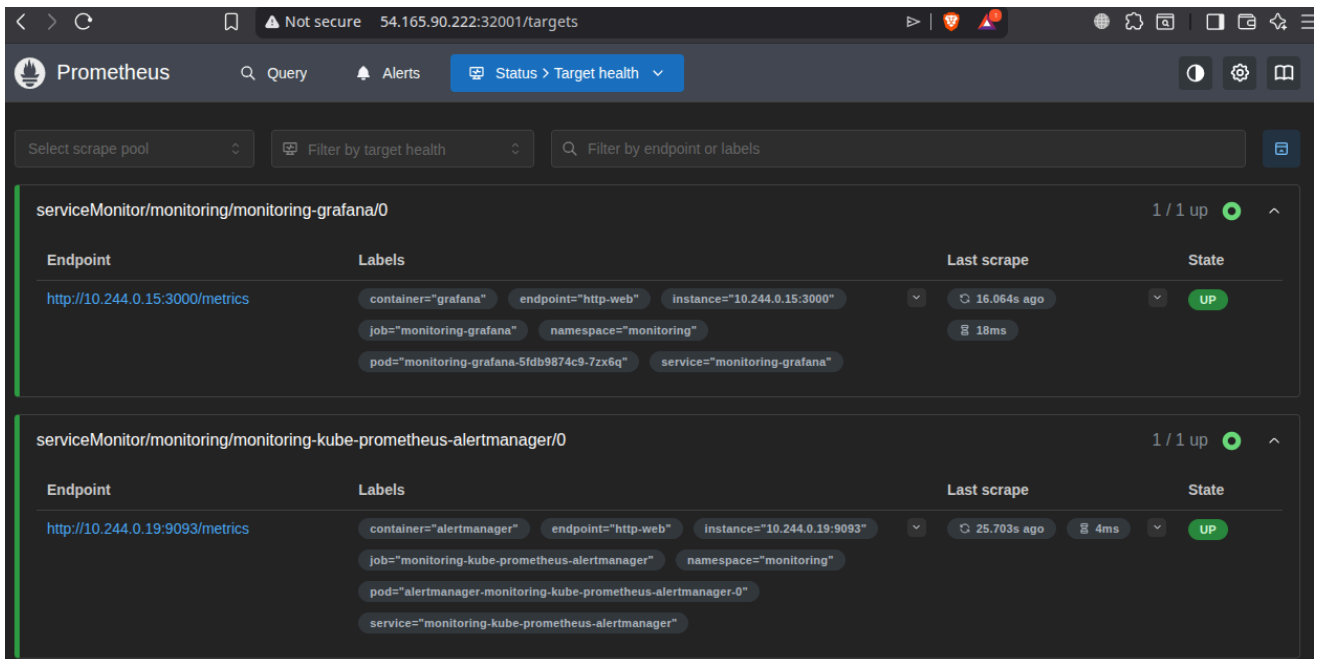


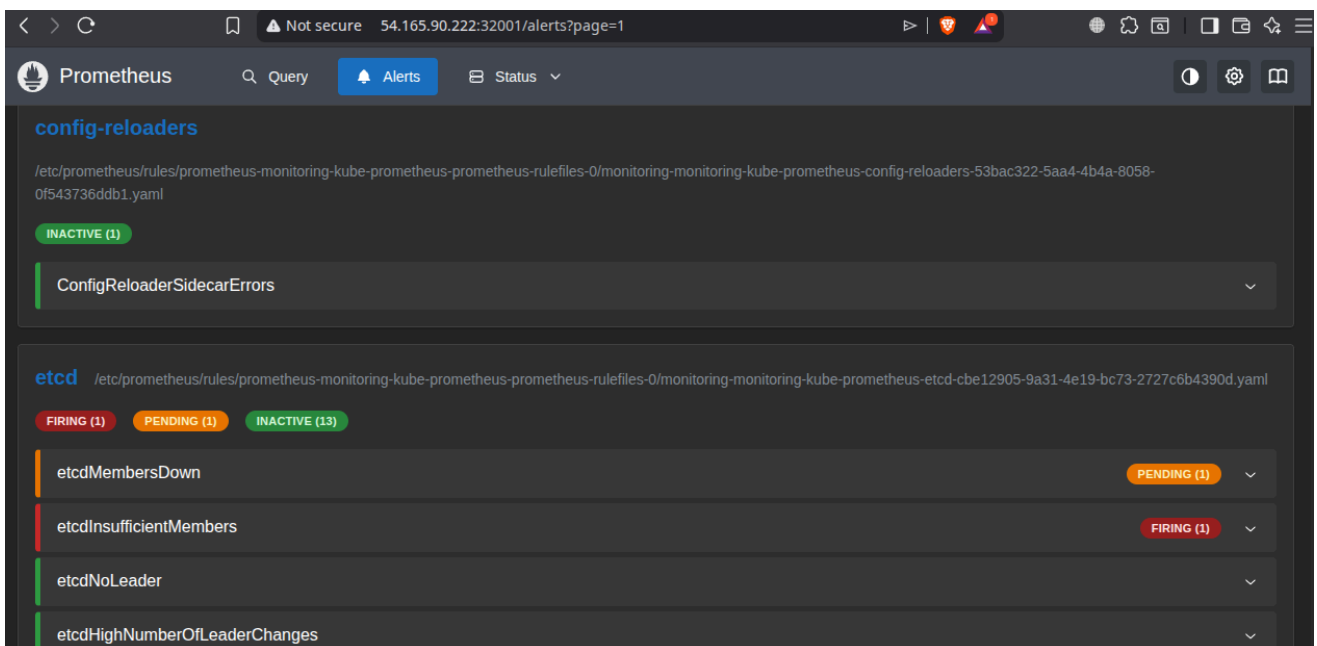
Figure 25 — Métriques API Server Kubernetes

6.5 Prometheus



Prometheus-Dashboard-Metrics.Png

Figure 26 — Interface Prometheus



Prometheus-Dashboard-Alerts.Png

Figure 27 — Alertes Prometheus

7. Résultats et Validation

7.1 Pipeline complet

Le pipeline CI/CD est pleinement opérationnel. À chaque push sur GitHub, les étapes suivantes sont exécutées automatiquement.

Étape	Résultat
Checkout	Clonage du dépôt
Build Docker Image	Construction de l'image
Push Docker Hub	Publication de l'image
Deploy Kubernetes	Rolling update
Verify Rollout	Vérification du déploiement

Le processus complet s'exécute en moins de **trois minutes**.

7.2 Validation du monitoring

La stack **Prometheus + Grafana** collecte correctement les métriques suivantes :

- utilisation CPU et mémoire des pods
- métriques de l'API Kubernetes
- état des nœuds et des pods
- latence et taux d'erreurs

7.3 Difficultés techniques

Défi	Solution
Accès kubectl depuis Jenkins	Tunnel SSH + kubeconfig flatten
Exposition NodePorts	Redirection de port avec socat
Certificats TLS	kubeconfig avec certificats intégrés

8. Conclusion

Ce projet démontre la mise en place d'un pipeline CI/CD complet automatisant le cycle de vie d'une application cloud-native.

L'architecture adoptée, basée sur deux instances EC2 distinctes, permet de séparer clairement les fonctions d'intégration continue et de déploiement. L'utilisation combinée de Jenkins, Docker et Kubernetes permet d'obtenir un système fiable, automatisé et reproductible.

Les principaux acquis du projet concernent la maîtrise des pipelines Jenkins, la conteneurisation avec Docker, le déploiement Kubernetes et la mise en place d'une

solution de monitoring avec Prometheus et Grafana.

Des améliorations futures pourraient inclure l'intégration de tests automatisés dans le pipeline, l'utilisation d'un cluster Kubernetes managé comme **Amazon EKS**, ou encore l'optimisation de l'exposition réseau de Minikube.